

VAJRA- Reengineering the legacy C++ compiler

Soumitra Chatterjee, Mahesha S, Shanthal Vasanth,
Mahesha Nanjundaiah, Shruti Doval

Technology Solutions Group

soumitra@hp.com, mahesha.s@hp.com, shanthal.vasanth@hp.com,
mahesha.nanjundaiah@hp.com, shruti@hp.com

Abstract

The most complex part in the maintenance of tightly coupled legacy software is re-engineering selected sub-components of the overall software infrastructure without destabilizing the stable components. A non-trivial re-engineering exercise becomes absolutely necessary when one or more subcomponents cannot be patched or enhanced incrementally anymore to make them more contemporary and adaptive to the dynamics of the ecosystem and ever changing demands of the user world. One such example has been the HP ANSI C++ compiler on the HP-UX PA-RISC platform. Typically, a compiler is a huge monolithic system software, with very tight coupling across sub-components. The reengineering initiative codenamed “VAJRA” was an effort to replace parts of the legacy compiler front-end with a robust and industry standard front-end from Edison Design Group^[2] (EDG), without regressing on external and internal interfaces. The end result is a robust, standard compliant ANSI C++ compiler, providing significant business benefits for HP by reducing HP-UX PA-RISC C++ compiler support investment with least disturbance to the huge install base. The key techniques from the VAJRA initiative can equally be applied to other C++ compilers for seamless reengineering.

Introduction

The HP ANSI C++ compiler^[1] on HP-UX PA-RISC platform, also known as HP PA aC++, has been in existence for almost a decade with many C++ application developers – both ISVs and end users – developing and shipping their products using aC++.

A C++ compiler must ensure release to release binary compatibility, also known as Application Binary Interface (ABI) compatibility, of the generated object file(s) in order to be acceptable to the developer community in the enterprise space. The C++ language itself requires that the compiler be extremely sensitive and strictly compliant to the ABI compatibility constraints. In order to support many of the C++ language constructs and their usage scenarios, the compiler generates a lot of facilitative and infrastructural support code in object files. These form the ABI elements of the particular C++ compiler, and are crucial in maintaining release to release ABI compatibility between the generated object files. Some examples of specific ABI elements that get generated by the C++ compiler are Object Layout (factoring in the class inheritance relationship), Virtual Tables (to facilitate virtual calls), Exception Tables and Handling Code, Constructors/Destructors, Global Static Initialization, Name Mangling, Pointer-to-members, etc.

As with most commercial compilers, the front-end of the HP aC++ compiler embeds the logic of determining the various factors leading to ABI compliance. A sub-component of the aC++ front-end, known as the “ABI Expander”, operates on the Abstract Syntax Tree (AST) generated by the aC++ parser, post semantic analysis, to generate the ABI elements, which are then passed on to the back-end in an intermediate representation called “Ucode”, for code generation.

Problem Statement

The HP PA-RISC aC++ compiler was originally developed with a front-end infrastructure more suited to the pre ANSI C++ Standard C++ language^[5]. The ANSI C++ standardization in 1998 made the language more flexible and versatile through the introduction of advanced C++ template features and more intuitive function overload resolution rules. This encouraged the widespread development of complex C++ programs using templates and

complex C++ constructs, which the HP PA aC++ compiler was unable to handle due to its outdated scanner, parser and semantic analyzer. Earlier attempts at patching to enhance the front-end to handle the complex ANSI C++ constructs introduced regressions and instability as the original front-end design was brittle and non-extensible. Continued attempts at enhancing the existing front-end to support the evolving language features such as template-template parameters, failed miserably due to the infrastructural flaws. The innumerable number of patches and attempted enhancements made on top of the fragile front-end made the compiler even more unstable, leading to a constant inflow of bugs, without any significant improvement in the support for language features. This continued to heavily impact the Total Customer Experience (TCE).

The above factors, coupled with the strong demand from the developer community for better support of the latest language features and the fact that it was in the best interest for HP to rapidly reduce investment in PA-RISC, fuelled the requirement to explore radical options in reengineering the HP PA aC++ compiler front-end infrastructure to make it more robust, extensible, and standard compliant, without in any way compromising on the compatibility requirements. At the same time, it should also allow for rapid reduction in the HP PA aC++ compiler support over the next 5 years, but require minimal investment towards implementing the solution itself.

Replacing the entire front-end with a new one connecting directly to the back-end had to be ruled out because of the compatibility constraints and the fact that the C++ ABI on HP-UX PA-RISC is not documented end to end, and there is no solid test infrastructure to certify that the new front-end adheres to ABI constraints. As such, the existing “ABI Expander” had to stay, and any solution to be implemented had to be seamlessly fit into it to ensure strict ABI compliance.

The Solution – VAJRA

Some of the most popular C++ compilers, such as the HP Integrity aC++ compiler, make use of a C++ language front-end designed and developed by EDG. The EDG language front-end is extremely stable, fully ANSI C++ standard compliant and extensible, enjoying a lot of developer credibility.

The solution, codenamed “VAJRA”, substitutes the current aC++ scanner, parser and semantic analyzer sub-components with those from the EDG front-end, carefully selecting the components that influence developer experience from the EDG front-end, and combining them with the components that affect ABI compliance (notably, the “ABI Expander”) from the existing front-end. All other phases of the EDG front-end, like low-level transformations (also known as “lowering”) and ABI generation are ignored by extracting the EDG Intermediate Language (IL) representation before it has undergone lowering.

A new “Bridge” sub-component was developed to translate the high-level EDG IL into the aC++ Abstract Syntax Tree (AST), fit for consumption by the existing “ABI Expander” sub-component of the legacy infrastructure, followed by code generation. Figure 1 below illustrates the approach.

The key innovation of the solution lies in the selective inclusion of ABI-neutral sub-components from the EDG front-end supplemented by the legacy front-end sub-components, and retrofitting the “un-lowered” EDG IL into the existing legacy downstream front-end sub-components for lowering and ABI generation, conformant to that produced by the existing infrastructure. Thus, the solution ensured absolute binary compatibility and ABI compliance by its very design.

Another notable advantage of this approach was that almost the entire existing infrastructure was reused, including the option processing and environment setup. The only new sub-component being developed from scratch was the “Bridge” layer, which being extremely thin, drastically limited the complexity of the approach and the scope of introducing new bugs. This ensured option compatibility as well, a significant feat as it meant absolutely no disturbance to the existing developer community, requiring no change to the existing build setups.

Also note that the responsibility of maintenance and evolving the front-end in line with the latest developments in the C++ language standards has shifted to EDG, thus significantly reducing the support cost for HP PA aC++.

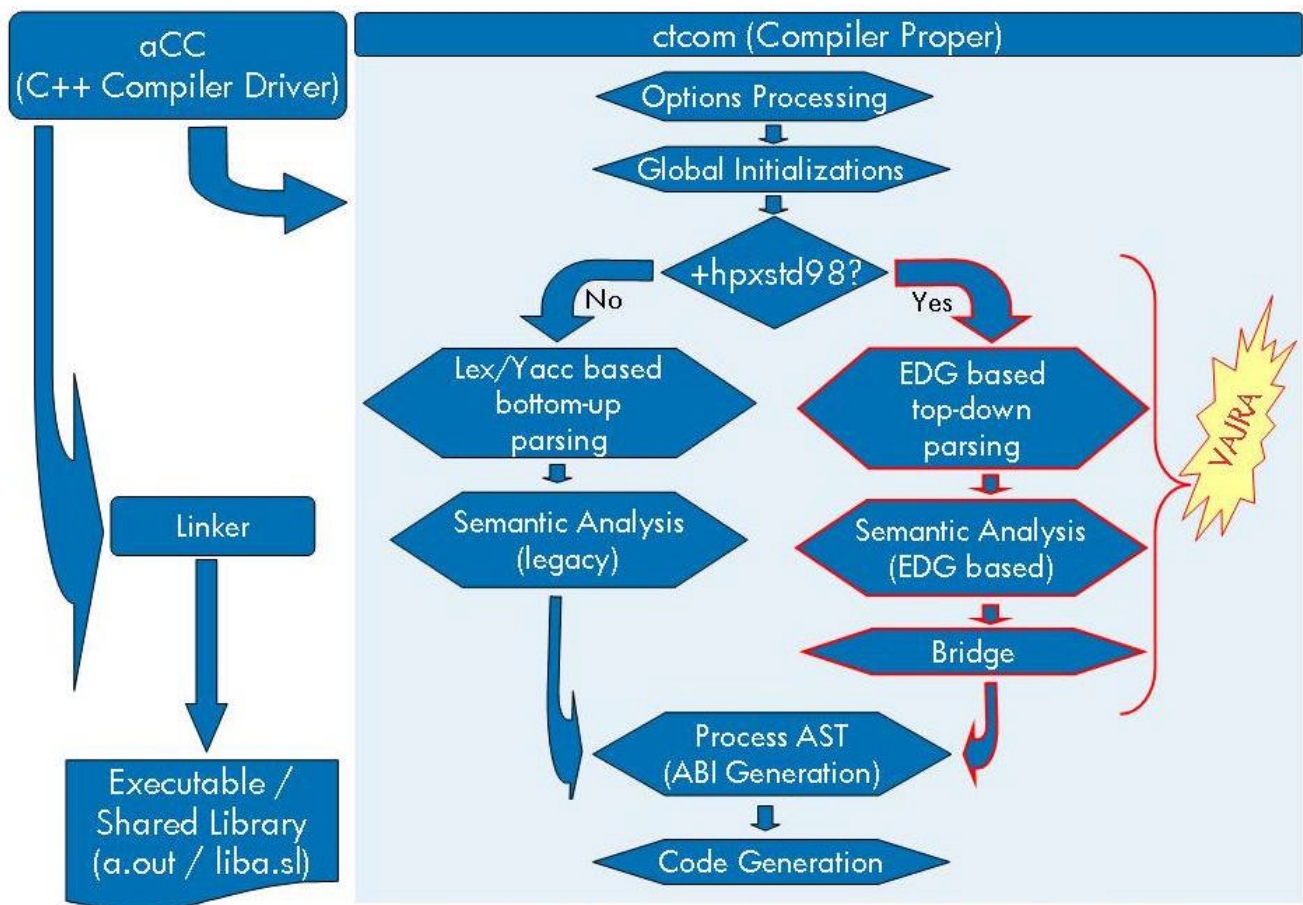


Figure 1. VAJRA - Reengineered HP PA aC++ compiler infrastructure

Evidence that the Solution Works and Current Status

The solution has been implemented as an optional feature of HP PA aC++ compiler version A.03.80 released through AR0709, enabled through the compile-time option “+hpxstd98”. The new features available with this option are as follows:

- Extensive support for templates including support for template-template parameters and popular C++ template libraries like Boost and Loki
- Faster application compile time as well as improved application runtime performance as observed with certain benchmarks and customer applications
- Improved standards compliance
- Improved compiler diagnostics
- Source and option parity, and a common source base for HP aC++ compilers on both PA-RISC and Integrity

This is in addition to the obsoleted existing defect backlog due to the above solution, saving about 200 EWs of effort. Overall, the solution drastically reduced the investment requirement for the HP PA aC++ compilers.

Competitive Approaches

Other approaches typically make use of the entire EDG front-end, including the lowering and ABI expansion phase, with a bridge layer to emit the intermediate representation accepted by the respective compiler back-end. This approach has been adopted in HP Integrity aC++ compiler and the Intel C++^[3] compilers, since both use the standard Integrity C++ ABI^[4], natively supported by the EDG front-end. The HP PA aC++ compiler supports a proprietary C++ ABI, and hence this approach would not work for HP PA-RISC. The VAJRA approach provides a very cost effective and safe alternative for such compilers having proprietary ABI requirements for similar reengineering.

Conclusions

The above solution, while drastically improving the TCE for HP customers, also promises to reduce the support engineering investment by at least 50% in the next 3-5 years. Sharing the front-end across both the PA-RISC and Integrity aC++ compilers provides further scope of investment reduction through synergy across the teams maintaining the respective compilers.

References

1. HP ANSI C++ compilers for PA-RISC and Integrity: <http://www.hp.com/go/cpp>
2. Edison Design Group: <http://www.edg.com>
3. Intel Compilers: <http://www.intel.com/cd/software/products/asmo-na/eng/compilers/284132.htm>
4. Integrity C++ ABI: <http://www.codesourcery.com/cxx-abi/abi.html>
5. C++ Standards: <http://www.open-std.org/jtc1/sc22/wg21/docs/standards>